

M1 MMS : Calcul Scientifique 2

TP3: Approximation de fonctions sur maillage 2D

Dans ce TP, on considère l'approximation polynomiale par morceaux sur une triangulation Ω_h d'un domaine ouvert $\Omega \subset \mathbb{R}^2$.

Pour cela, on définit des fonctions polynomiales de degré k sur chaque triangle du maillage : $u_K : K \rightarrow \mathbb{R}$ puis on reconstruit l'approximation globale par morceaux $u_h : \Omega_h \rightarrow \mathbb{R}$ telle que $u_h|_K := u_K$ pour chaque élément K .

On suppose disposer d'une base de polynômes sur les noeuds d'un triangles notée $\{\phi_k\}_{k=0,\dots,r}$ pour $r+1$ degrés de liberté. Ainsi l'approximation locale peut s'écrire $u_K(x) = \sum_{k=0}^r c_k \phi_k(x)$. Les coefficients c_k permettent d'encoder l'approximation. Par exemple $c_k = u(x_k)$ pour l'interpolation. L'approximation globale est construite par combinaison de l'ensemble des fonctions de base définies sur l'ensemble des $n + 1$ noeuds du maillage: $u_h(x) = \sum_{j=0}^n c_j \phi_j(x)$.

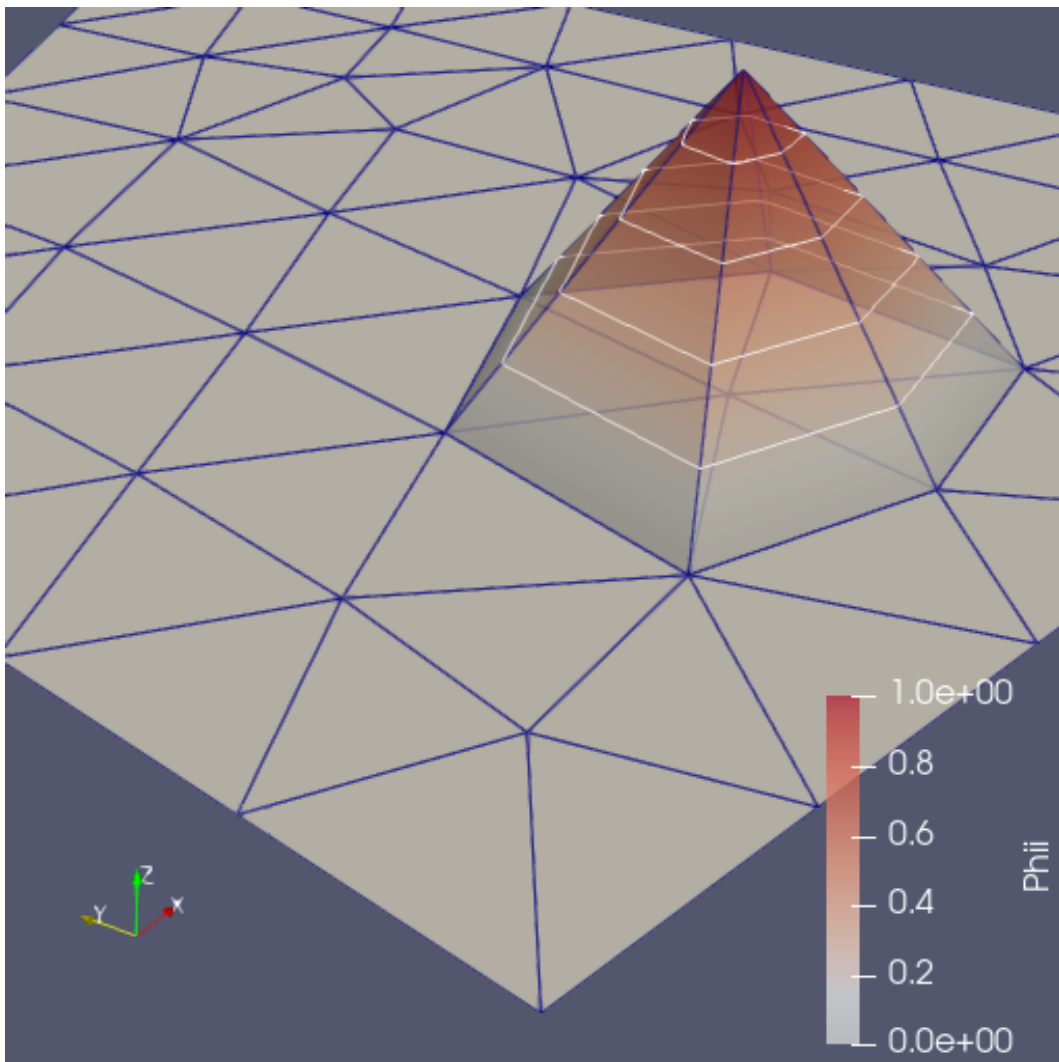


Figure 1: Une fonction de base de degré 1

Formules d'intégration dans les triangles: Une formule d'intégration à n_q points sur un élément K consiste en les deux vecteurs de dimension q des poids, w_q et des points, ξ_q tels que:

$$\int_K f(x)dx \simeq \sum_{q=0}^{n_q-1} w_q f(x_q)$$

Quelques formules sont données dans la figure 2.

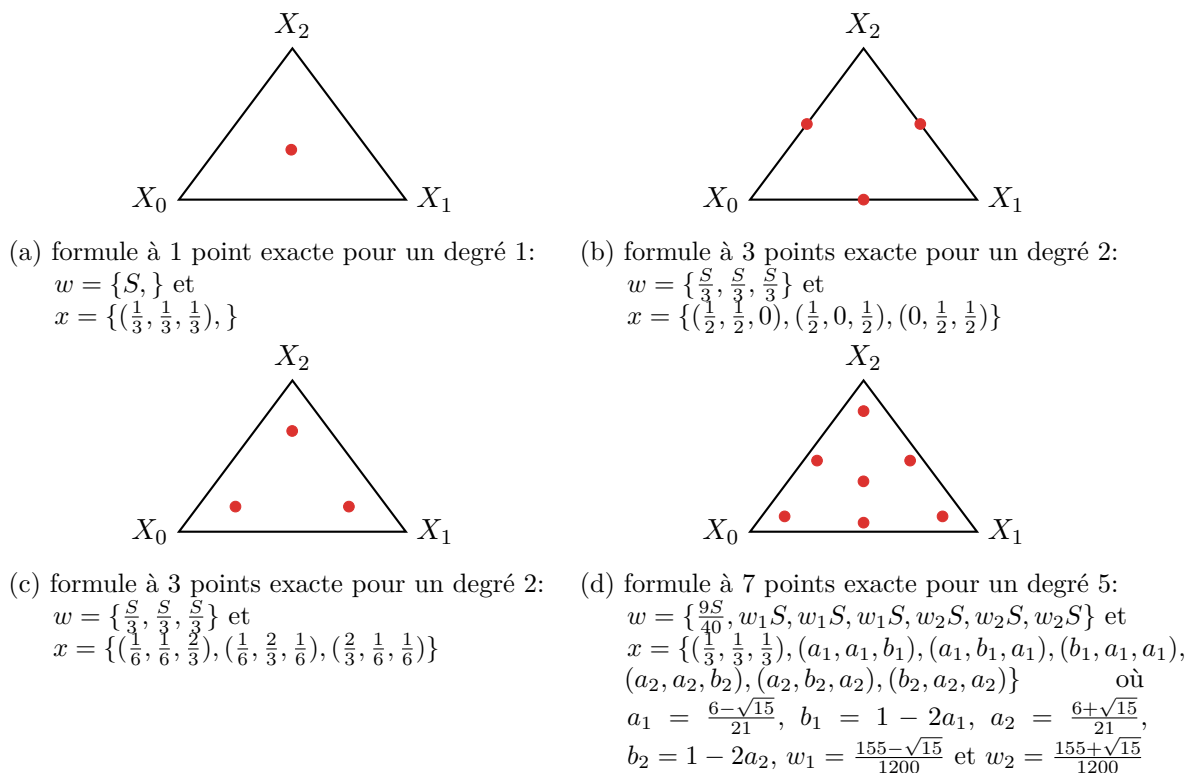


Figure 2: Quelques formules d'intégration numérique sur les triangles (données en coordonnées barycentriques et pour un triangle d'aire S).

Exercice 1. Projection constante par morceaux

Sur un maillage d'un domaine $\Omega \subset \mathbb{R}^2$ **non rectangulaire** de votre choix Ω_h , calculer une projection orthogonale sur $\pi_h^0(f) \in P^0(\Omega_h)$ pour une fonction f de votre choix définie sur Ω et à valeurs dans $\mathbb{R}$. L'espace d'approximation est de taille égale au nombre de triangles du domaine car on fixe un seul degré de liberté par triangle. Le plus simple est de considérer le barycentre du triangle. Les fonctions de base $\{\phi_j\}_j$ sont définies de la manière suivante pour un triangle K_j du maillage:

$$\phi_j(x) = 1_{K_j} = \begin{cases} 1 & \text{si } x \in K_j \\ 0 & \text{sinon} \end{cases}$$

L'inconnue $\pi_h^0(f)$ est cherchée sous la forme de ses coordonnées dans la base des $\{\phi_j\}_j$.

1. Donner le système linéaire à résoudre.
2. Décrire la structure de la matrice.
3. Simplifier le système de sorte à obtenir une expression explicite des c_j dans le cas particulier d'une formule d'intégration à un point.

4. En utilisant les résultats des questions précédentes, implémenter une fonction Python permettant de calculer la projection, dans le cas d'une formule d'intégration à un point. Le résultat est calculé sous la forme d'un tableau numpy 1D. L'écriture du résultat dans un fichier peut se faire à l'aide du module `meshio` de la manière suivante:

```

1 meshio.xdmf.write(
2     "Exercice1.xdmf",
3     meshio.Mesh(
4         points=np.reshape(mesh.getNodes()[1], (-1,3)),
5         cells=[("triangle", np.reshape(mesh.getElementsByType(2)[1], (-1,3))-1)],
6         cell_data={'ProjectionP0': [projP0]})

```

On notera l'utilisation de l'argument `cell_data` qui permet de donner une valeur par cellule (ou élément ici). L'argument attend un objet Python de type `dict[str, list[ArrayLike]]`

La visualisation se fait à l'aide du logiciel de visualisation `Paraview`. Dans ce logiciel, il suffit d'ouvrir le fichier `Exercice2.xdmf` avec le mode `XDMF Reader`. On pourra visualiser le maillage avec l'option "Surface with edges", ajouter des filtres (par exemple `Contour`), ...

Exercice 2. Projection linéaire par morceaux

Sur un maillage d'un domaine quadrangulaire Ω_h , calculer une approximation de degré 1 par projection orthogonale $L^2(\Omega)$ notée $\pi_h^1(f) \in P^1(\Omega_h)$. L'espace d'approximation est de taille égale au nombre de noeuds du domaine car on fixe 3 degrés de liberté par triangle. Les fonctions de base sont celles de la base de Lagrange sur les triangles. Pour un noeud j appartenant à un triangle K :

$$\phi_j(x) = \begin{cases} \lambda_k^K(x) & \text{si } x \in \overline{K} \text{ et } j = \text{idx}(K, k) \\ 0 & \text{sinon} \end{cases}$$

où $\lambda_k^K(x)$ est la k -ième coordonnée barycentrique associée à l'élément K et $\text{idx}(K, k)$ l'indice du k -ième noeud du triangle K .

1. Donner le système linéaire à résoudre et décrire la structure de la matrice dans le cas où la formule à 1 point de la figure 2 est utilisée.
2. À partir d'un maillage généré par `GMSH`, calculer la projection. Le résultat est calculé sous la forme d'un tableau numpy 1D dont l'ordre des valeurs est le même que celui des noeuds du maillage. L'écriture du résultat dans un fichier peut se faire à l'aide du module `meshio` de la manière suivante:

```

1 meshio.xdmf.write(
2     "Exercice2.xdmf",
3     meshio.Mesh(
4         points=np.reshape(mesh.getNodes()[1], (-1,3)),
5         cells=[("triangle", np.reshape(mesh.getElementsByType(2)[1], (-1,3))-1)],
6         point_data={'ProjectionP1': projP1})

```

On notera l'utilisation de l'argument `point_data` qui permet de donner une valeur par noeud. L'argument attend un objet Python de type `dict[str, ArrayLike]`

Exercice 3. Amélioration de l'implémentation

Produire une nouvelle version de votre programme de l'exercice précédent qui implémente les points suivants:

- Les formules de quadratures peuvent être modélisées par une classe, sur le modèle de la classe suivante, pour la formule à trois points:

```

1 class Q3pt():
2     def __init__(self):
3         self.weights = [1/3, 1/3, 1/3]
4         self.points = [np.array((0.5, 0.5, 0.)),
5                         np.array((0.5, 0., 0.5)),
6                         np.array((0., 0.5, 0.5))]

```

- Un calcul générique du système linéaire peut être implémenté, à partir d'une fonction f , d'un maillage mesh et d'une formule de quadrature q . Une telle fonction implémente l'algorithme suivant:

```

1 def projectionP1(f, mesh, q):
2     triangles = np.reshape(mesh.getElementsByType(2)[1], (-1,3))
3     NNodes = mesh.getNodes()[0].size
4     # Allocations matrice et vecteur
5     ...
6     for ie, elem in enumerate(triangles):
7         # Récupération des coordonnées du triangle et calcul de l'aire du triangle
8         ...
9         for w, p in zip(q.weights, q.points):
10             # p,w dans l'élément de référence, et p en coordonnées barycentriques
11             # Calcul du point de quadrature dans l'élément réel x:=x(p)
12             ...
13             for dofi, i in enumerate(elem-1):
14                 # Pour chaque degré de liberté local 'dofi' associé à l'indice global i
15                 ...
16                 for dofj, j in enumerate(elem-1):
17                     # Pour chaque degré de liberté local 'dofj' associé à l'indice global j
18                     ...
19     proj=scipy.linalg.solve(A,b)
20     return proj

```

1. Implémenter quelques formules d'intégration numérique sur le modèle de la classe `Q3pt`. On pourra implémenter des formules illustrées sur la figure 2
2. Implémenter le calcul de la projection en utilisant l'algorithme générique donné ci-dessus.

Exercice 4. Convergence en maillage

Sur la base de l'exercice précédent, réaliser une étude de convergence en maillage pour la projection sur $P^1(\Omega_h)$.

Indications:

- Pour la projection sur $P^1(\Omega_h)$ on pourra créer une fonction générique pour le calcul de l'erreur $L_2(\Omega_h)$ dont la signature et l'algorithme est le suivant:

```

1 def errorL2P1(f, uh, mesh, q):
2     triangles = np.reshape(mesh.getElementsByType(2)[1], (-1,3))
3     error = 0.
4     ...
5     for ie, elem in enumerate(triangles):
6         # Récupération des coordonnées du triangle et calcul de l'aire du triangle
7         ...

```

```

8   for w, p in zip(q.weights, q.points):
9       # p, w dans l'élément de référence, et p en coordonnées barycentriques
10      # Calcul du point de quadrature dans l'élément réel x:=x(p)
11      ...
12      error += ... # (w_q)*(f(x_q)-uh(x_q)) ** 2
13  return sqrt(error)

```

- Pour évaluer $u_h(x_q)$, on pourra utiliser une boucle `for dofi, i in enumerate(elem-1):` afin de parcourir chaque degré de liberté local `dofi` associé à l'indice global `i` d'un élément $\mathcal{K}$.

Exercise 5. Matrices creuses

Les matrices issues des méthodes numériques (différences finies, éléments finis, ...) sont généralement creuses (avec seulement quelques coefficients non nuls). La taille des matrices est en $\mathcal{O}(n^2)$, avec n le nombre d'inconnues. Le stockage des matrices denses empêche d'augmenter n , en particulier dans des dimensions plus grandes (2D, 3D, ...). On utilise alors une structure de données dédiée aux matrices creuses : seuls les coefficients non nuls et la structure de la matrice sont stockés. Cela permet des économies significatives de mémoire ainsi qu'une réduction du temps de calcul grâce à des algorithmes appropriés.

Il est à noter que l'inverse d'une matrice creuse n'est généralement pas creux. Ainsi, on utilise généralement le stockage des matrices creuses en combinaison avec des algorithmes itératifs. L'algorithme de calcul de l'inverse a une complexité de $\mathcal{O}(n^3)$, tandis que les méthodes itératives ont une complexité de $\mathcal{O}(kn^2)$ pour k itérations, et se réduisent à $\mathcal{O}(kN)$ pour des matrices creuses contenant N coefficients non nuls.

Le format *Compressed Sparse Row* (CSR) est l'un des formats les plus courants pour stocker des matrices creuses générales. Une matrice de taille n est représentée à l'aide de trois tableaux contenant :

- Les coefficients non nuls (tableau de taille N , généralement $N \ll n^2$)
- Les indices de colonnes des coefficients non nuls (tableau d'entiers de taille N)
- La taille cumulative des lignes en termes de coefficients non nuls (tableau d'entiers de taille $n + 1$, commençant par 0).

Par exemple la matrice suivante

$$\begin{pmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 0 \\ 2 & 0 & 1 & 0 \\ -2 & 0 & 0 & 1 \end{pmatrix}$$

est créée au format creux comme suit:

```

1  import numpy as np
2  from scipy.sparse import csr_matrix
3  # Dense matrix
4  A = np.asarray((( 1, 0, 2, 0),
5                   ( 0, 1, 0, 0),
6                   ( 2, 0, 1, 0),
7                   (-2, 0, 0, 1)))
8  # Convert to sparse matrix
9  sA = csr_matrix(A)
10 print(sA.data)      # non zero coefficients
11 print(sA.indices)  # column indices of coefficients
12 print(sA.indptr)   # rows structure

```

La sortie du programme est:

```
[ 1  2  1  2  1 -2  1]
[0 2 1 0 2 0 3]
[0 2 3 5 7]
```

La structure CSR peut être construite lors du calcul des contributions. Pour cela on peut remarquer que la structure est construite ligne par ligne. Ainsi on peut créer une liste de longueur égale au nombre de noeuds du maillage constituée de dictionnaires dont les clés sont les numéros de colonne et les valeurs les coefficients de la matrice. L'intérêt est de pouvoir créer la structure de la matrice en même temps que le calcul de ses éléments. Par exemple:

```
1 ARowdata = [{}, ] * 4
2 ARowdata[0] = {0: 1., 2: 2}
3 ARowdata[1] = {1: 1., }
4 ARowdata[2] = {0: 2., 2:1}
5 ARowdata[3] = {0:-2, 3:1}
6 A=sparse.csr_matrix([(d for line in ARowdata for d in line.values()),
7                       [j for line in ARowdata for j in line.keys()],
8                       np.cumsum([0,] + [len(line) for line in ARowdata]))])
```

Le format CSR est bien adapté au stockage de matrice creuses utilisées pour calculer des produits avec un vecteur. Un autre format est plus simple à gérer pour assembler la matrice, sans assembler la matrice dense. Il s'agit du format COO. Pour cela, on construit les 3 tableaux de même taille contenant les valeurs des coordonnées et des coefficients tels que $A[i[k], j[k]] = data[k]$. Par exemple:

```
1 row = np.array([0, 0, 1, 2, 2, 3, 3])
2 col = np.array([0, 2, 1, 0, 2, 0, 3])
3 data = np.array([1, 2, 1, 2, 1, -2, 1])
4 coo_matrix((data, (row, col)))
```

Il est à noter que les indices dupliqués sont sommés, ce qui est très pratique dans notre cas.

1. Adapter la fonction `projectionP1` afin de passer la matrice à la fonction `matplotlib.pyplot.spy`. Qu'observez-vous ?
2. Ré-écrire la fonction de projection sur P^1 en assemblant directement une matrice creuse. Validez votre implémentation à l'aide d'un test numérique.